

Doc. No.: WG21/P0668R0

Date: 2017-6-19

Reply-to: Hans-J. Boehm

Email: hboehm@google.com

Authors: Hans-J. Boehm, Olivier Giroux, Viktor Vafeiades, with input from Will Deacon, Doug Lea, Daniel Lustig, Paul McKenney and others

Audience: SG1

P0668R0: Revising the C++ memory model

Although the current C++ memory model, adopted essentially in C++11, has served our user community reasonably well in practice, a number of problems have come to light. The first one of these is particularly new and troubling:

1. Existing implementation schemes on Power and ARM are not correct with respect to the current memory model definition. These implementation schemes can lead to results that are disallowed by the current memory model when the user combines acquire/release ordering with seq_cst ordering. On some architectures, especially Power and Nvidia GPUs, it is expensive to repair the implementations to satisfy the existing memory model. Details are discussed in (Lahav et al) <http://plv.mpi-sws.org/scfix/paper.pdf> (which this discussion relies on heavily). The same issues were briefly outlined at the Kona SG1 meeting. We summarize below.
2. Our current definition of memory_order_seq_cst, especially for fences, is too weak. This was caused by historical assumptions that have since been disproven.
3. We still do not have an acceptable way to make our informal (since C++14) prohibition of out-of-thin-air results precise. The primary practical effect of that is that formal verification of C++ programs using relaxed atomics remains infeasible. The above paper suggests a solution similar to <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3710.html> . We continue to ignore the problem here, but try to stay out of the way of such a solution.
4. The current definition of memory_order_consume is widely acknowledged to be unusable, and implementations generally treat it as memory_order_acquire. The current draft "temporarily discourages" it. See <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0371r1.html> . There are proposals to repair it (cf. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0190r3.pdf> and <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0462r1.pdf>), but nothing that is ready to go.

Here we concentrate on, and outline proposals for, the first two, and merely keep the last two in mind.

Power, ARM, and GPU implementability

Although we previously believed otherwise, it has recently been shown that the standard implementations of memory_order_acquire and memory_order_release on Power are insufficient. Very briefly, these are compiled using "lightweight" fences, which are insufficient to enforce required properties for memory_order_seq_cst accesses to the same location.

To illustrate, we borrow the Z6.U example, from S 2.1 of <http://plv.mpi-sws.org/scfix/paper.pdf> (pseudo-C++ syntax, memory orders indicated by subscript, e.g. $y =_{\text{rel}} 1$ abbreviates $y.\text{store}(\text{memory_order_release}, 1)$):

Thread 1	Thread 2	Thread 3
$x =_{\text{sc}} 1$	$b = \text{fetch_add}(y)_{\text{sc}} // 1$	$y =_{\text{sc}} 3$
$y =_{\text{rel}} 1$	$c = y_{\text{rlx}} // 3$	$a = x_{\text{sc}} // 0$

The comments here indicate the observed assigned value.

The indicated outcome here is disallowed by the current standard: All `memory_order_seq_cst` (sc) accesses must occur in a single total order, which is constrained to have $a = x_{\text{sc}} // 0$ before $x =_{\text{sc}} 1$ (since it doesn't observe the store), which must be before $b = \text{fetch_add}(y)_{\text{sc}} // 1$ (since it happens before it), which must be before $y =_{\text{sc}} 3$ (since the `fetch_add` does not observe the store, which is forced to be last in modification order by the load in Thread 2). But this is disallowed since the standard requires the happens before ordering to be consistent with the sequential consistency ordering, and $y =_{\text{sc}} 3$, the last element of the sc order, happens before $a = x_{\text{sc}} // 0$, the first one.

On the other hand, this outcome is allowed by the Power implementation. Power normally uses the "leading fence" convention for sequentially consistent atomics. (See <http://www.cl.cam.ac.uk/~pes20/cpp/cpp0xmappings.html>) This means that there is only an `lwsync` fence between Thread 1's instructions. This does not have the "cumulativity"/transitivity properties that would be required to make the store to x visible to Thread 3 under these circumstances.

This issue was missed in earlier analyses. This example is not a problem for the "trailing fence" mapping that could also have been used on Power. But Lahav et al contains examples that fail for that mapping as well, for similar reasons.

This example relies crucially on the fact that a `memory_order_release` operation synchronizes with a `memory_order_seq_cst` operation on the same location. Code that consistently accesses each location only with `memory_order_seq_cst`, or only with weaker ordering, is not affected, and works correctly.

Whether or not such code occurs in practice depends on coding style. One reasonable coding style is to initially use only `seq_cst` operations, and then selectively weaken those that are performance critical; it does result in such cases. Even in such cases, it seems clear that the current compilation strategy does not result in frequent failures; this problem was discovered through careful theoretical analysis, not bug reports. It is unclear whether there is any real code that can fail as a result of the current mapping; it would require careful analysis of the use cases to determine whether the weaker ordering provided by the hardware is in fact sufficient for these use cases.

For ARM, the situation is theoretically similar, but appears to be much less severe in practice. On ARMv8, the usual compilation mode for loads and stores is currently to compile acquire/release operations as `seq_cst` operations, so there is currently no issue. On ARMv7, some compilation schemes for acquire/release have the same issues as for Power, but the most common scheme seems to be to use "dmb ish", which does not share this problem.

Nvidia GPUs have a memory model similar to Power, and share the same issues, probably with larger cost differences. For very large numbers of cores, it is natural to share store buffers between threads, which may make stores visible to different threads in inconsistent orders. This lack of "multi-copy atomicity" is also the core distinguishing property of the Power and ARM memory models. We suspect that other GPUs are also affected, but cannot say that definitively.

We are not aware of issues on other existing CPU architectures. Since it appears more attractive to drop multi-copy atomicity with higher core counts, we expect the same issue to recur with some future processor designs.

Alternative 1: "Just" fix the implementations

Repairing this on Power without changing the specification would prevent us from generating the lighter weight "lwsync" fence instruction for a `memory_order_release` operation (unless we either know it will never synchronize with a `memory_order_seq_cst` operation, or we make `memory_order_seq_cst` operations even more expensive), which would have a significant performance impact on acquire/release synchronization. It would also defeat a significant part, though not all, of the motivation for introducing `memory_order_acquire` and `memory_order_release` to start with.

The cost on GPUs is likely to be higher.

Among people we informally surveyed, this was not a popular option. Many people felt that we would be penalizing a subset of the available machine architectures for an issue with little practical impact. The language would no longer be able to express pure acquire-release synchronization, which many people feel is essential.

We could regain acquire-release synchronization by adding a new "weak" atomic type that does not support `memory_order_seq_cst`, and requires explicit `memory_order` arguments. Again there was general concern that this significantly increases the library API footprint for an issue without much practical impact.

Alternative 2: Fix the standard

This is the approach taken by Lahav et al, and the one we pursue here.

The proposal in Lahav et al is mathematically elegant. Currently the standard requires that the sequential consistency total order S is consistent with the happens before relation. Essentially, if any two sc operations are ordered by happens before, then they must be ordered the same way by S . In our example, this requires $x =_{sc} 1$ to be ordered before $b = \text{fetch_add}(y)_{sc} // 1$ in spite of the fact that the hardware mapping does not sufficiently enforce it. The core fix (S1fix in the paper) is to relax the restriction that a happens before ordering implies ordering in S to only the sequenced before case, or the case in which the happens before ordering between a and b is produced by a chain

a is sequenced before x happens before y is sequenced before b

The downside of this is that "happens before" now has a rather strange meaning, since sequentially consistent operations can appear to execute in an order that's not consistent with it.

In the Z6.U example, $x =_{sc} 1$ must no longer precede $b = \text{fetch_add}(y)_{sc} // 1$ in the sequential consistency order S, in spite of the fact that the former "happens before" the latter. And in the questionable execution that we now wish to allow, they indeed have the opposite order in S.

We propose to make this somewhat less confusing by suitably renaming the relations in the standard as follows:

Currently the initialization rules, etc. use "strongly happens before" in guaranteeing ordering. The current intent is to also use that to specify library ordering, such as for mutexes. We propose to modify that definition to require "sequenced before" ordering at both ends. This new improved "strongly happens before" would be used in the same contexts as now, and would remain strong enough to ensure that if a happens before b , and they both participate in the sc ordering S, then a also precedes b in S. "Strongly happens before" would continue to exclude any ordering via `memory_order_consume`, since such ordering is much more restrictive, and must be explicitly accommodated by the caller.

Thus we would propose to change 4.7.1p11 [intro.races, a.k.a. 1.10] roughly as follows (this is an early attempt at wording, which is still under discussion):

An evaluation A ~~strongly~~ simply happens before an evaluation B if either

- A is sequenced before B, or
- A synchronizes with B, or
- A simply happens before X and X simply happens before B.

[*Note:* In the absence of consume operations, the happens before and ~~strongly~~ simply happens before relations are identical. ~~Strongly~~ Simply happens before essentially excludes consume operations. It is used only in the definition of strongly happens before below. — end note]

An evaluation A strongly happens before an evaluation D if, either

- A is sequenced before D, or
- A synchronizes with D, and both A and D are sequentially consistent atomic operations (32.4 [atomics.order]), or
- there are evaluations B and C such that A is sequenced before B, B simply happens before C, and C is sequenced before D, or
- there is an evaluation B such that A strongly happens before B, and B strongly happens before D.

[*Note:* Informally, if A strongly happens before B, then A appears to be evaluated before B in all contexts. --end note]

We would then adjust 32.4p3 [atomics.order] correspondingly:

There shall be a single total order S on all `memory_order_seq_cst` operations, consistent with the "strongly happens before" order, and modification orders for all affected locations, such that each `memory_order_seq_cst` operation B that loads a value from an atomic object M observes one of the following values: ...

and add a second note at the end of p3:

[Note: We do not require consistency with "happens before" (4.7.1 [intro.races]). This allows more efficient implementation of memory_order_acquire and memory_order_release on some machine architectures. It may produce more surprising results when these are mixed with memory_order_seq_cst accesses. -- end note]

Strengthen SC fences

The current `memory_order_seq_cst` fence semantics do not guarantee that a program with only `memory_order_relaxed` accesses and `memory_order_seq_cst` fences between each pair actually exhibits sequential consistency. This was, at one point, intentional. The goal was to ensure that architectures like Itanium that allow stores to become visible to different processors in different orders, and do not provide fences to rectify this, could be supported. But it subsequently became clear that Itanium, as a result of failing to provide strong ordering for accesses to a single location) would need to use stronger primitives for `memory_order_relaxed` anyway. All known SC fence implementations provide the stronger semantics, and we should acknowledge that.

We propose to strengthen the `memory_order_seq_cst` fence semantics as suggested in Lahav et al. (wording TBD)

Straw Polls

1. Should we change the standard to accommodate architectures (Power, GPU, ARM to some extent) to allow outcomes such as the Z6.U outcome discussed above?
2. Is this the right direction for doing so?
3. Should we clean up and strengthen `seq_cst` fences along the above lines?